3DGEngine Scripting Guide

Shared game module, gameplay logic, runtime services, and AI scripts

Updated for the new game/ module architecture

Write a script once and run it in both Editor Play mode and the standalone player.

Contents

Update summary
Gameplay and behavior-tree scripts now belong under game/. The editor and player both link the same static library and call RegisterGameModule(). The older player-only registration workflow is now only a thin forwarding layer.

Old approach	Current approach
Put gameplay sources in player/src	Put reusable scripts in game/include/game/scripts and game/src/scripts
Register in player/src/GameScripts.cpp	Register once in game/src/GameModule.cpp
Editor and player registrations could diverge	Both hosts link and register the same game module

1. The shared game module

The new game library is the project-level gameplay layer, similar to a game module in a larger engine. It depends on engine, while both 3DGEditor and player depend on game. This keeps engine reusable and keeps project-specific logic in one place.

```
3DGEngine/
  engine/                       reusable engine systems
  game/
    CMakeLists.txt              shared static library
    include/game/GameModule.h   registration entry point
    include/game/scripts/       gameplay and BT headers
    src/GameModule.cpp          one registration list
    src/scripts/                optional implementations
  editor/                       links engine + game
  player/                       links engine + game
```

Startup host	Registration path
Editor	RegisterExampleBtScripts, legacy RegisterGameBtScripts, then RegisterGameModule
Player	RegisterGameScripts forwards to RegisterGameModule

Single source of truth
Add reusable project scripts to game/src/GameModule.cpp. Do not duplicate the same registration in editor and player.

2. Quick start: use the included Spinner

Spinner is a header-only example already registered by the game module. It rotates its object around world Y every frame.

```
// game/include/game/scripts/Spinner.h
class Spinner : public engine::Script {
public:
    void OnUpdate(float dt) override {
        if (auto* t = Transform()) {
            const glm::quat spin = glm::angleAxis(
                dt * 1.5f, glm::vec3(0.0f, 1.0f, 0.0f));
            t->rotation = glm::normalize(spin * t->rotation);
        }
    }
};
```

- Select any scene object.
- Enable its Script component.
- Set Script Class to Spinner exactly.
- Enter Play mode and confirm rotation.
- Export the scene and run the player; the same class runs because player links game.

Why this example matters

It demonstrates the complete path: the class derives from engine::Script, GameModule includes it, ScriptRegistry registers the name, the scene stores that name, and both hosts instantiate it.

3. Create a gameplay script

Use a header-only class for small scripts, or split larger scripts into a header and source file.

Header

```
// game/include/game/scripts/DoorScript.h
#pragma once
#include <engine/gameplay/Script.h>

class DoorScript final : public engine::Script {
public:
    void OnCreate() override;
    void OnUpdate(float dt) override;
private:
    bool m_open = false;
};
```

Source

```
// game/src/scripts/DoorScript.cpp
#include "game/scripts/DoorScript.h"
#include <GLFW/glfw3.h>

void DoorScript::OnCreate() {
    m_open = GetFieldBool("startsOpen", false);
}

void DoorScript::OnUpdate(float dt) {
    if (WasKeyPressed(GLFW_KEY_E)) m_open = !m_open;
    if (auto* t = Transform()) {
        const float target = m_open ? 3.0f : 0.0f;
        t->position.y += (target - t->position.y) * dt * 5.0f;
    }
}
```

> **Include paths**
> Because game exposes its include directory publicly, use #include "game/scripts/DoorScript.h" from GameModule and other targets.

4. Register and compile scripts

Step A - list non-header-only sources

```
# game/CMakeLists.txt
add_library(game STATIC
    src/GameModule.cpp
    src/scripts/DoorScript.cpp
    src/scripts/MagicCombatScript.cpp
)
target_include_directories(game PUBLIC ${CMAKE_CURRENT_SOURCE_DIR}/include)
target_link_libraries(game PUBLIC engine)
```

Step B - include and register once

```
// game/src/GameModule.cpp
#include "game/scripts/Spinner.h"
#include "game/scripts/DoorScript.h"
#include "game/scripts/MagicCombatScript.h"

void RegisterGameModule() {
    auto& scripts = engine::ScriptRegistry::Instance();
    auto& bt = engine::ai::BtScriptRegistry::Instance();

    scripts.Register("Spinner", [] { return std::make_unique<Spinner>(); });
    scripts.Register("DoorScript", [] { return std::make_unique<DoorScript>(); });
    scripts.Register("MagicCombatScript", [] {
        return std::make_unique<MagicCombatScript>();
    });
}
```

Step C - rebuild

```
cmake -S . -B build -DBUILD_TESTING=ON
cmake --build build --config Debug --target 3DGEditor player
```

> **Exact-name rule**
> The registration string must exactly match the Script Class stored by the scene. Runtime diagnostics report missing classes by object name.

5. Attach scripts and expose fields

- Select the object in Hierarchy.
- Enable Script in the Inspector or add it from the component Add menu.
- Set the class name registered by GameModule.
- Add typed Float, Int, Bool, or String fields.
- Save and re-export the scene after changing fields.

Read fields with safe defaults

```
const float damage = GetFieldFloat("damage", 25.0f);
const int burst = GetFieldInt("burstCount", 16);
const bool enabled = GetFieldBool("enabled", true);
const std::string target = GetFieldString("target", "Enemy");
```

Field type	Typical values
Float	speed, damage, cooldown, radius
Int	score reward, burst count, lives
Bool	starts open, invulnerable, autoplay
String	target name, asset path, next scene

Attach once, run everywhere

The scene only stores the class name and fields. The editor and player create the same compiled class from the shared registry.

6. Lifecycle, input, objects, and components

Callback	Use
OnCreate	One-time lookup and initialization.
OnUpdate	Frame input, decisions, animation/audio/camera commands, timers.
OnFixedUpdate	Physics movement and deterministic simulation.
OnDestroy	Cleanup before entity or scene removal.

Input and object access

```
if (WasKeyPressed(GLFW_KEY_F)) { /* one press edge */ }
if (IsKeyDown(GLFW_KEY_W))     { /* held */ }

const auto staff = FindObject("MagicStaff");
auto* staffTransform = FindTransform("MagicStaff");
auto* myTransform = Transform();

if (auto* hp = TryGet<engine::Health>(staff)) hp->Damage(10.0f);
```

Component helpers

- TryGet() and Has() operate on Self.
- TryGet(entity) and Has(entity) inspect another object.
- Add() and Remove() change components on Self.
- Destroy and DestroySelf queue safe removal after script iteration.

> **Scheduling rule**
> Use OnFixedUpdate for physics work. Read frame-edge input and mouse deltas in OnUpdate.

7. Complete magic staff attack

This shared-module script plays a full-body action, triggers staff effects, delays projectile release, clones a configured prototype, and fills its Projectile component.

```
void MagicCombatScript::OnUpdate(float dt) {
    m_cooldown = std::max(0.0f, m_cooldown - dt);
    if (!WasKeyPressed(GLFW_KEY_F) || m_cooldown > 0.0f ||
        IsAnimationActionPlaying()) return;

    m_cooldown = GetFieldFloat("cooldown", 0.8f);
    PlayAnimationProfile("StaffAttack");
    PlayAudio(m_staff, true);
    BurstParticles(m_staff, GetFieldInt("burstCount", 18));
    ShakeCameraAdvanced(0.08f, 1.5f, 0.25f, 22.0f, 0.5f);
    Delay(GetFieldFloat("releaseDelay", 0.28f),
          [this] { ReleaseFireball(); });
}

void MagicCombatScript::ReleaseFireball() {
    auto* source = Transform();
    if (!source) return;
    const glm::vec3 forward = source->rotation * glm::vec3(0,0,1);
    const auto fireball = SpawnFromObject(
        GetFieldString("projectile", "FireProjectilePrototype"),
        source->position + glm::vec3(0,1.1f,0) + forward * 0.8f);
    if (auto* p = TryGet<engine::Projectile>(fireball)) {
        p->dir = glm::normalize(forward);
        p->owner = Self();
        p->damage = GetFieldFloat("damage", 30.0f);
        p->speed = GetFieldFloat("projectileSpeed", 14.0f);
    }
}
```

Module setup
Place MagicCombatScript.h under game/include/game/scripts, its .cpp under game/src/scripts, list the source in game/CMakeLists.txt, then register it in GameModule.cpp.

8. Animation actions and events

Need	Call
Full-body action	PlayAnimationAction or PlayAnimationProfile
Layered action	PlayMaskedAnimationAction with root bone
State float/bool/trigger	SetAnimationParameter, SetAnimationBool, SetAnimationTrigger
Queries	GetAnimationParameter, GetAnimationBool, IsAnimationActionPlaying
Movement lock	IsAnimationMovementLocked
Clip event	WasAnimationEvent

```
if (!IsAnimationActionPlaying()) {
    PlayAnimationProfile("HeavyAttack");
}
if (WasAnimationEvent("StaffImpact")) {
    BurstParticles(FindObject("ImpactEmitter"), 24);
}
```

Non-layered actions

Full-body actions block player and AI movement until completion. Masked actions preserve locomotion while the selected bone branch plays.

Profile recommendation

Store clip name/index, fade, speed, and mask in an Animation Action Profile. Scripts call the stable profile name instead of repeating tuning values.

9. Audio, particles, cameras, and triggers

Audio

```
PlayAudio(staff, true); PauseAudio(staff); ResumeAudio(staff);
SetAudioVolume(staff, 0.75f); SetAudioPitch(staff, 1.1f);
SetAudioAttenuation(staff, 1.0f, 35.0f, 1.0f);
ApplyAudioSnapshot(engine::AudioSnapshotPreset::Combat, 0.4f);
LoadAdaptiveMusic("Content/Audio/Battle.music");
SetMusicState("Combat", true);
```

Particles and camera

```
PlayParticles(emitter, true); BurstParticles(emitter, 30);
SetParticleRate(emitter, 80.0f); SetParticleSpeed(emitter, 1.25f);
ShakeCamera(1.0f, 0.3f, 20.0f);
PlayCameraSequence("BossReveal", true, true);
if (WasCameraSequenceEvent("BossReveal", "BossVisible")) { /* begin */ }
```

Triggers

```
const auto player = FindObject("PlayerStart");
if (WasTriggerEntered(player)) PlayCameraSequence("EnterTemple");
if (IsTriggerTouching(player)) {
    if (auto* hp = TryGet<engine::Health>(player)) hp->Damage(5.0f * dt);
}
```

10. Runtime services

Service	Purpose
SpawnEmpty	Create a named object with Transform.
SpawnFromObject	Clone a named scene prototype and place it.
SetTimer / ClearTimer	Repeating callback on the update loop.
Delay	One-shot callback.
SaveValue / LoadValue	Persistent string key/value data.
SaveCheckpoint / LoadCheckpoint	Persistent named world position.
RequestSceneLoad	Safe queued runtime scene transition.

```
int pulse = SetTimer(1.0f, [this] { BurstParticles(5); }, true);
Delay(5.0f, [this, pulse] { ClearTimer(pulse); });

SaveValue("player.staffLevel", "3");
SaveCheckpoint("TempleEntrance", Transform()->position);

if (WasTriggerEntered(FindObject("PlayerStart"))) {
    RequestSceneLoad("Levels/TempleInterior.3dgscene");
}
```

Safe scene changes
RequestSceneLoad is consumed after the current script update, so registry teardown never happens inside a script callback.

11. Behavior-tree scripts in game/

Custom tasks, decorators, and services also belong in the shared game module. Derive from engine::ai::BtScript and register through BtScriptRegistry in GameModule.cpp.

```
class StaffStrikeTask final : public engine::ai::BtScript {
public:
    engine::ai::BtStatus Tick(engine::ai::AgentContext& c, float) override {
        if (!c.registry || c.targetEntity == engine::ecs::kNull)
            return engine::ai::BtStatus::Failure;
        if (glm::length(c.targetPos - c.agent.position) > c.reachRadius)
            return engine::ai::BtStatus::Failure;
        if (auto* hp = c.registry->TryGet<engine::Health>(c.targetEntity)) {
            hp->Damage(20.0f);
            return engine::ai::BtStatus::Success;
        }
        return engine::ai::BtStatus::Failure;
    }
};

bt.Register("StaffStrikeTask", [] {
    return std::make_unique<StaffStrikeTask>();
});
```

- Task/service: override Tick and return Running, Success, or Failure.
- Decorator: override Check and return whether the branch may run.
- Use c.blackboard for shared typed data.
- Use c.registry, c.self, and c.targetEntity for ECS access.
- The registered name must match the class selected in the behavior graph.

12. Migration, debugging, and packaging

Move old player-only scripts

- Move headers to game/include/game/scripts.
- Move .cpp files to game/src/scripts.
- Add .cpp files to game/CMakeLists.txt.
- Move registration lines to game/src/GameModule.cpp.
- Remove duplicate editor/player registrations.
- Keep player/src/GameScripts.cpp as the existing forwarder.

Troubleshooting

Symptom	Check
Missing class warning	Registration string and GameModule include.
Link error for script methods	Source listed in game/CMakeLists.txt.
Works in player but not editor	Editor links game and calls RegisterGameModule; rebuild editor.
Fields use defaults	Inspector field name/type matches code.
Spawn returns null	Prototype object name exists in exported scene.
Old behavior after change	Rebuild both target and re-export scene where fields changed.

Packaging

The game module is statically linked into player, so no separate gameplay DLL is required. Package runtime scenes and referenced content as usual.

> **Legacy registration**
> Editor still calls legacy RegisterGameBtScripts for compatibility. New project gameplay and BT scripts should use the shared game module.

13. API quick reference

Group	Helpers
Core	Self, Registry, Transform, FindObject, FindTransform, TryGet, Has, Add, Remove
Lifetime	Destroy, DestroySelf, SpawnEmpty, SpawnFromObject
Input/events	key and mouse state, trigger enter/touch/exit, animation events
Animation	actions, masked actions, profiles, state parameters, movement lock
Audio	transport, cursor, volume/pitch, bus, spatial controls, snapshots, adaptive music
Particles	play, stop, restart, burst, clear, enable, rate, speed, count
Camera	shake, sequences, finish/skip/timeline events
Runtime	timers, delay, save values, checkpoints, scene requests
Fields	GetFieldString, GetFieldFloat, GetFieldInt, GetFieldBool

Files to remember

File	Role
game/include/game/scripts/*.h	Script classes
game/src/scripts/*.cpp	Script implementations
game/src/GameModule.cpp	All gameplay and BT registrations
game/CMakeLists.txt	Compile non-header-only sources
engine/include/engine/gameplay/Script.h	Exact gameplay API signatures
player/src/GameScripts.cpp	Thin forwarder to RegisterGameModule

> **Recommended workflow**
> Create in game/, register once, compile editor and player, attach the exact class name, test Editor Play, export, then test standalone player.

End of updated guide